



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 02

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 2

GRUPO DE TEORÍA: 6

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 26/08/25

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- generar las figuras copiando los vértices de triángulo rojo y cuadrado verde:

- triángulo azul
- triángulo verde (0,0.5,0)
- cuadrado rojo
- cuadrado verde
- cuadrado café (0.478, 0.255, 0.067)

```
//Triángulo azul
GLfloat vertices_triánguloazul[] = {
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
    0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
    0.0f, 0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
};

MeshColor* triánguloazul = new MeshColor();
triánguloazul->CreateMeshColor(vertices_triánguloazul, 18);
meshColorList.push_back(triánguloazul);

//Triángulo verde
GLfloat vertices_triánguloverde[] = {
    -0.5f, -0.5f, 0.5f, 0.0f, 0.5f, 0.0f,
    0.5f, -0.5f, 0.5f, 0.0f, 0.5f, 0.0f,
    0.0f, 0.5f, 0.5f, 0.0f, 0.5f, 0.0f,
};

MeshColor* triánguloverde = new MeshColor();
triánguloverde->CreateMeshColor(vertices_triánguloverde, 18);
meshColorList.push_back(triánguloverde);

GLfloat vertices_cuadradoverde[] = {
    //X      Y      Z      R      G      B
    -0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    -0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
};

MeshColor* cuadradoverde = new MeshColor();
cuadradoverde->CreateMeshColor(vertices_cuadradoverde, 36);
meshColorList.push_back(cuadradoverde);

// Cuadrado café
GLfloat vertices_cuadradocafe[] = {
    -0.5f, -0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    0.5f, -0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    0.5f, 0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    -0.5f, 0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    -0.5f, -0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    0.5f, -0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    -0.5f, 0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
    0.5f, 0.5f, 0.5f, 0.478f, 0.255f, 0.067f,
};

MeshColor* cuadradocafe = new MeshColor();
cuadradocafe->CreateMeshColor(vertices_cuadradocafe, 36);
meshColorList.push_back(cuadradocafe);

// Cuadrado rojo
GLfloat vertices_cuadradorojo[] = {
    -0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
};

MeshColor* cuadradorojo = new MeshColor();
cuadradorojo->CreateMeshColor(vertices_cuadradorojo, 36);
meshColorList.push_back(cuadradorojo);
```

Como ya teníamos la lógica para crear la figura con colores lo único que se hizo fue replicar para todo lo que necesitamos y los únicos cambios fueron el nombre del arreglo más aparte cambiar rgb para el color que necesitamos y para finalizar lo convertimos a un objeto de Meshcolor

después guardamos en la lista meshColorList para posteriormente poder llamar al objeto.

2.- Usando la proyección ortogonal generar el siguiente dibujo a partir de instancias de las figuras anteriormente creadas , recordar que todos se dibujan en el origen y por transformaciones geométricas se desplazan.

```
shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

// cuadrado rojo -> rectángulo
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -0.40f, -4.0f));
model = glm::scale(model, glm::vec3(0.85f, 1.10f, 1.0f)); // ancho, alto
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[6]->RenderMeshColor(); // cuadrado rojo

// triángulo azul
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.55f, -4.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.80f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[2]->RenderMeshColor(); // triángulo azul

// Ventana izquierda (cuadrado verde)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.20f, -0.10f, -3.0f));
model = glm::scale(model, glm::vec3(0.30f, 0.30f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor(); // cuadrado verde

// Ventana derecha (cuadrado verde)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.20f, -0.10f, -3.0f));
model = glm::scale(model, glm::vec3(0.30f, 0.30f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor(); // cuadrado verde

// Puerta cuadrado verde lo pase a rectangulo
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -0.75f, -3.0f));
model = glm::scale(model, glm::vec3(0.30f, 0.40f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor(); // cuadrado verde
```

```

model = glm::scale(model, glm::vec3(0.30f, 0.40f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[4]->RenderMeshColor(); // cuadrado verde

// para el arbol izquierdo

// triángulo verde
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.75f, -0.45f, -4.0f));
model = glm::scale(model, glm::vec3(0.45f, 0.60f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[3]->RenderMeshColor(); // triángulo verde

// Tronco cuadrado café
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.75f, -0.85f, -4.0f));
model = glm::scale(model, glm::vec3(0.20f, 0.20f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[5]->RenderMeshColor(); // cuadrado café

// arbol derecho

// triángulo verde
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.75f, -0.45f, -4.0f));
model = glm::scale(model, glm::vec3(0.45f, 0.60f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[3]->RenderMeshColor(); // triángulo verde

//cuadrado café
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.75f, -0.85f, -4.0f));
model = glm::scale(model, glm::vec3(0.20f, 0.20f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[5]->RenderMeshColor(); // cuadrado cafe

glUseProgram(0);
mainWindow.swapBuffers();

```

Primero lo que hacemos es en `shaderList[1].useShader();` usamos el índice 1 para poder hacer uso de las letras, posteriormente Inicializamos la matriz para cada una de nuestras figuras, después con traslación movemos nuestras figuras (el eje x derecha e izquierda, el eje y arriba y abajo, la coordenada z nos sirve para ver que figura queda enfrente de otra por así decirlo), posteriormente la siguiente línea envía la matriz al shader y al final se dibuja la figura, el índice que recibe es la posición que ocupa dentro de nuestro vector `meshColorList` y esto se debe a que cada vez que se llama a `CrearLetrasyFiguras()`, creamos figuras de colores y las vamos agregando al vector con `push_back`. El orden en que se insertan determina el índice que después usamos para poder llamarlas.

Estas instrucciones se repitieron para todas las figuras que se utilizaron para armar la casa, ya únicamente era de ver en que coordenadas mover las figuras para tener las formas y en los vectores de crear figuras ver tamaños y como las inicializamos y de igual forma con ayuda del scale ver que tan ancha y alta tenía que estar la figura para que hubiera coherencia con la imagen de referencia.

Ejecución



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código.

Ningún problema con el código.

3. Conclusión:

- a. Los ejercicios de la clase: Complejidad, explicación

Este ejercicio si estuvo un poco mas complejo, este me costo un poco más de trabajo identificar que hacer, aunque en realidad fue sencillo una vez que entendí que se debía de mover, al momento si me costo trabajo al punto en que me revolví con el código.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

Considero que si faltó un poco más de explicación de cómo funcionaba el código implementado, porque la explicación teórica todo bien, si bien los comentarios que puso fueron de bastante ayuda en el momento por tanto código como que me revuelvo, por tanto.